

INSTITUTO FEDERAL
Sul-rio-grandense

Câmpus
Pelotas



CRUD COM LARAVEL

CRUD Usando o Eloquent ORM

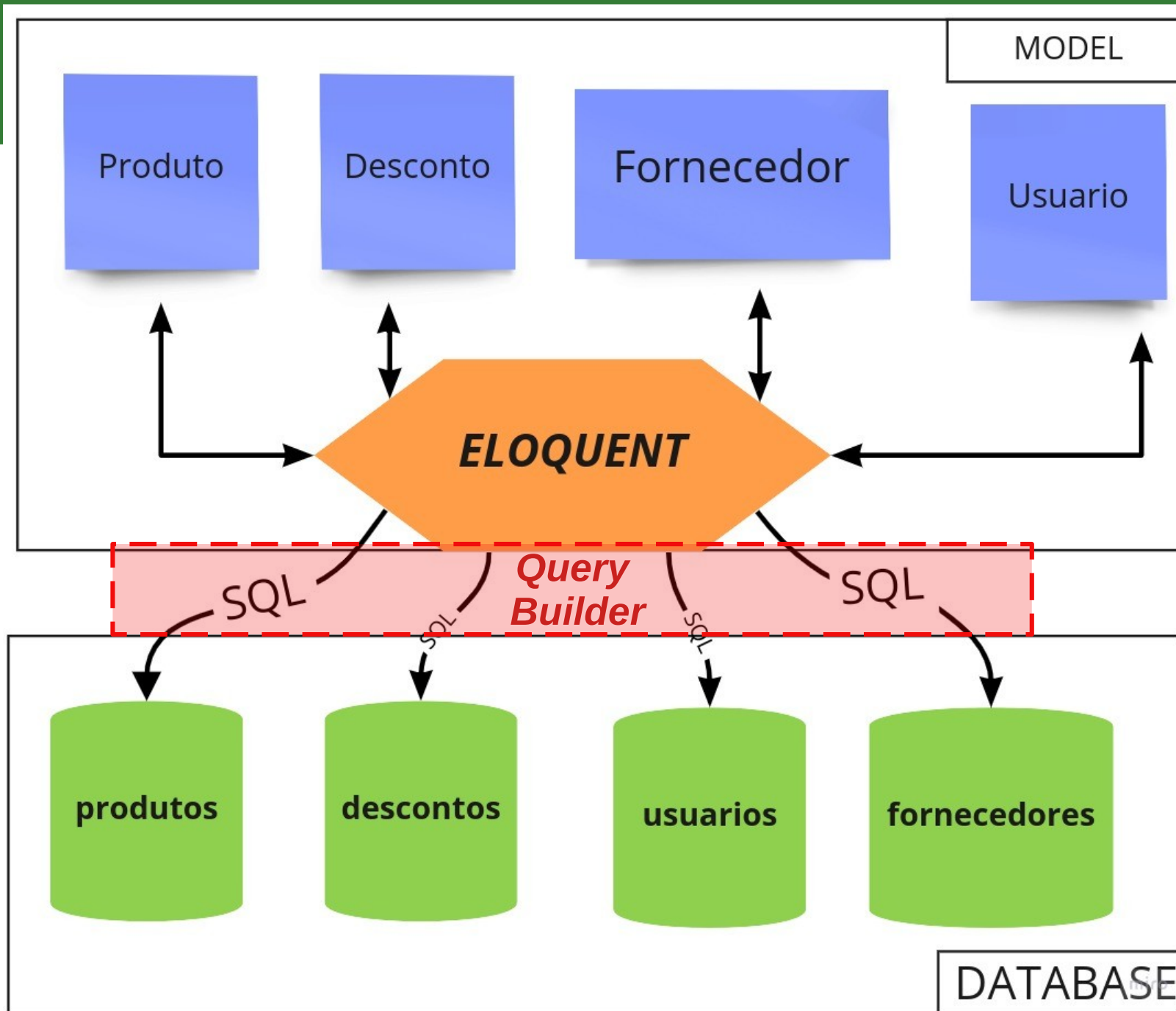
CSTSI - V SEMESTRE
DESENVOLVIMENTO DE APLICAÇÕES ORIENTADAS A OBJETOS
Prof. Gill Velleda Gonzales | gillgonzales@ifsul.edu.br
Julho, 2024



- **INTRODUÇÃO AO ORM ELOQUENT**
- **REVISÃO AULA 05 (FUNCIONALIDADE READ)**
- **DESENVOLVIMENTO DE UM CRUD**
 - **Implementando a funcionalidade CREATE**
 - Rotas para a página de criação
 - Método no Controlador
 - *Lendo uma Requisição (Classe Request)*
 - Página de criação na View (Template Blade)
 - *Trabalhando com formulários html (CSRF)*
 - **Implementação da funcionalidade UPDATE**
 - **Implementação da funcionalidade DELETE**
 - Criação de página de confirmação para DELETE
- **ATIVIDADE PRÁTICA**



- Representa a camada de mapeamento **Objeto-Relacional** do *framework Laravel*.
- Todas as Models usam o **Eloquent**;
- Não é necessário escrever o SQL;
all(), *find()*, *findOrFail()*, etc...
 - ***all()***: retorna uma EloquentCollection, representando todos os registros do banco de dados;
 - ***find(id)***: retorna um objeto model representando um registro único de acordo com o parâmetro de chave-primária passado como argumento (id).
 - ***findOrFail(id)***: mesmo que o find, porém se não encontrar gera uma exceção
- Existem métodos para montar **SQL** através do **Query Builder**;
where(), *orderBy()*, etc... (Estes veremos nas próximas aulas)





- **Funcionalidade READ**

Na última aula já desenvolvemos esta funcionalidade, foi a última atividade prática. Basicamente a **READ** do **CRUD** consiste da recuperação e apresentação dos registros do banco de dados para uma determinada *Tabela (produtos)*.

- Criamos a classe **Produto** (Modelo)
 - `app/Models/Produto.php`
- Desenvolvemos o **ProdutoController**:
 - `app/Http/Controllers/ProdutoController.php`
 - Métodos **Index()**
- Implementamos a camada de View
 - Templates Blade:
 - `resources/views/produtos.blade.php` (Lista os registros Link.)
 - `resources/views/produto.blade.php` (Mostra um registro Link.)

CRUD com Laravel

Revisão da última aula – Funcionalidade READ do CRUD



- **Funcionalidade READ:** Criação da classe Produto (Modelo)

```
$> php artisan make:model Produto -mc
```

- Com o comando *artisan* foram criadas também a *migration (m)* e o *controller (c)*.

```
app > Models > Produto.php > ...
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Database\Eloquent\Model;
7
8  class Produto extends Model
9  {
10     use HasFactory;
11 }
12
```

CRUD com Laravel

Revisão da última aula – Funcionalidade READ do CRUD



- **Funcionalidade READ:** Configurações no banco de dados
 - Antes da criação da tabela **produtos** foi realizada a configuração do **DB**
 - Alteradas as variáveis de conexão do arquivo **.env**

DB_DATABASE; DB_USERNAME; DB_PASSWORD

```
> storage
> tests
> vendor
.editorconfig
.env
.env.example
.gitattributes

10
11 DB_CONNECTION=mysql
12 DB_HOST=127.0.0.1
13 DB_PORT=3306
14 DB_DATABASE=firstlaravel
15 DB_USERNAME=root
16 DB_PASSWORD=r00t
17
```


CRUD com Laravel

Revisão da última aula – Funcionalidade READ do CRUD



- **Funcionalidade READ:** Criação das migrations
 - A migration sempre segue o padrão de nome do **modelo** no **plural em snake_case** : **Produto** → **create_produtos_table.php**
 - As colunas da tabela do banco foram definidas no método **up**

```
bootstrap
> config
> database
  > factories
  > migrations
    2014_10_12_000000_create_users_table.php
    2014_10_12_100000_create_password_resets_table....
    2019_08_19_000000_create_failed_jobs_table.php
    2019_12_14_000001_create_personal_access_tokens...
    2022_02_23_143118_create_produtos_table.php
  > seeders
  .gitignore
  lang

13
14
15
16
17
18
19
20
21
22
23
24
25
26

*/
public function up()
{
    Schema::create('produtos', function (Blueprint $table) {
        $table->id();
        $table->text('nome');
        $table->text('descricao');
        $table->integer('qtd_estoque');
        $table->float('preco');
        $table->boolean('importado')->default(false);
        $table->timestamps();
    });
}
```

- Após, com o comando **artisan** se criou a tabela no banco para **Produto**:

```
$> php artisan migrate:install $> php artisan migrate:fresh
```


CRUD com Laravel

Revisão da última aula – Funcionalidade READ do CRUD



- **Funcionalidade READ:** Criação do controlador *ProdutoController*
- Com o comando `-mc` o artisan criou a migration e o *controlador*
 - `/app/Http/Controllers/ProdutoController.php`

The screenshot shows an IDE with a dark theme. On the left, the 'EXPLORER' panel displays the project structure of 'FIRSTLARAVEL'. The 'app' directory is expanded, showing 'Http' > 'Controllers'. 'ProdutoController.php' is selected. The main editor area shows the code for 'ProdutoController.php'. The breadcrumb at the top of the editor reads: 'app > Http > Controllers > ProdutoController.php > PHP Intelephense > ProdutoController'. The code is as follows:

```
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use App\Models\Produto;
6  use Illuminate\Http\Request;
7
8  class ProdutoController extends Controller
9  {
10
11     private $produto;
12
13     public function __construct()
14     {
15         $this->produto = new Produto();
16     }
```

CRUD com Laravel

Revisão da última aula – Funcionalidade READ do CRUD



- **Funcionalidade READ:** Criação da view *produtos.blade.php*
 - Criação do template que receberá o resultado da consulta no banco
 - */resources/views/produtos.blade.php*

The screenshot shows an IDE with the Explorer panel on the left and the editor on the right. The Explorer panel shows the project structure for 'FIRSTLARAVEL', including folders like 'lang', 'public', 'resources', 'css', 'js', 'views', and 'routes'. The 'views' folder is expanded, showing 'produtos.blade.php' and 'welcome.blade.php'. The editor displays the code for 'produtos.blade.php', which is a Blade template for displaying a list of products. The code includes a loop to iterate over the products and render them in a table.

```
22 <tbody>
23     @foreach($produtos as $produto)
24     <tr>
25         <td><a href="/produto/{{ $produto->id }}">
26         <td>{{ $produto->nome }}</td>
27         <td>{{ $produto->qtd_estoque }}</td>
28         <td>{{ $produto->preco }}</td>
29         <td>{{ ($produto->Importado) ? 'Sim' : 'Não' }}</td>
30     </tr>
31 @endforeach
```

CRUD com Laravel

Revisão da última aula – Funcionalidade READ do CRUD



- **Funcionalidade READ:** Criação do método *index()*
 - O método *index* usa o modelo para recuperar os registros da tabela.
 - Então repassa para a *view* pela função *view('produtos',['produtos'=>...])*

```
app > Http > Controllers > ProdutoController.php > ...
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use App\Models\Produto;
6
7  class ProdutoController extends Controller
8  {
9
10     public function index(){
11         return view('produtos',['produtos'=>Produto::all()]);
12     }
13
14 }
15
```

CRUD com Laravel

Revisão da última aula – Funcionalidade READ do CRUD



- **Funcionalidade READ:** Revisando a função (*helper*) **view**:
 - Usada para chamar o motor do Blade a renderizar um template;
 - Podemos passar dados as views por **arrays associativos**.
 - **view**('nome_do_template', ["chave1"=>'valor1', "chave2"=>'valor2']);
 - **helper**: funções usadas no framework para facilitar alguns processos
 - Ex: *redirect, route, dd, response etc ...*;
 - **Template**: nome do arquivo **.blade.php** na pasta **/resources/views/**;
 - **Chave**: nome da variável a ser recuperada no template da view;
 - **Valor**: valor da variável repassada á view;

```
10 public function index(){
11     return view('produtos', ['produtos'=>Produto::all()]);
12 }
```


CRUD com Laravel

Revisão da última aula – Funcionalidade READ do CRUD



- **Funcionalidade READ:** Recuperando a variável na **view**:
 - No template a variável é recuperada com o mesmo nome da chave do array associativo passado para a função **view()**.

```
FIRSTLARAVEL  resources > views > produtos.blade.php > html > body > table > tbody > tr > td
> js
> views
  produtos.blade.php
  welcome.blade.php
> routes
  api.php
  channels.php
  console.php
  web.php
> storage
> tests
> vendor
.editorconfig
.env
.env.example
.gitattributes

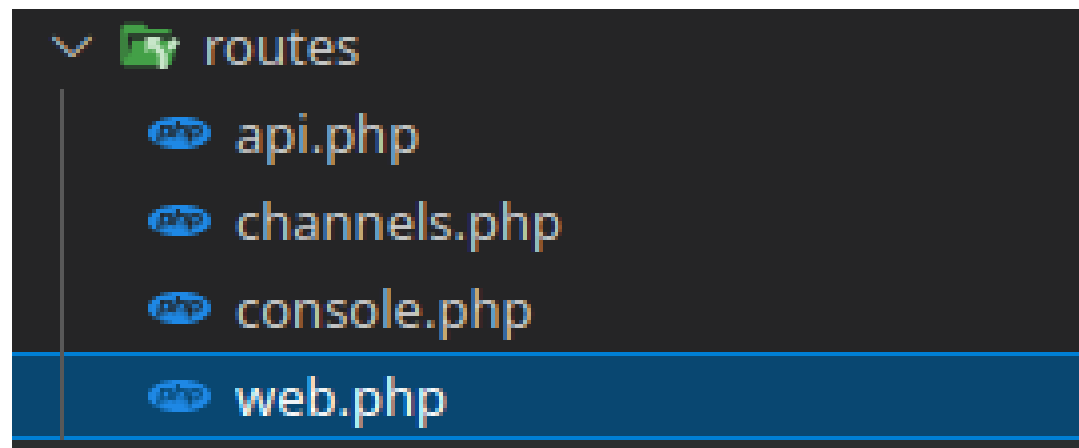
21      </thead>
22      <tbody>
23          @foreach($produtos as $produto)
24              <tr>
25                  <td>{{$produto->id}}</td>
26                  <td>{{$produto->nome}}</td>
27                  <td>{{$produto->qtd_estoque}}</td>
28                  <td>{{$produto->preco}}</td>
29                  <td>{{($produto->Importado)?'Sim':'Não'}}</td>
30              </tr>
31          @endforeach
32      </tbody>
```

CRUD com Laravel

Revisão da última aula – Funcionalidade READ do CRUD



- **Funcionalidade READ:** Criação da *rota* para a página de produtos
 - Agora só falta a criação da rota que invocará o método ***index*** do controlador.
 - O controlador então acessa os dados do ***modelo*** e repassa à ***view***
 - Como vimos no slide anterior no método ***index*** da classe ***ProdutoController***



```
26 Route::get('/ola',[HomeController::class,'index']);
27 Route::get('/produtos',[ProdutoController::class,'index']);
28
```



localhost:8001/produtos

Produtos

Id	Nome	qtd_estoque	preco	Importado
111	Samsung A5 - 2017	2	4500	Não
112	Notebook DELL Inspiron 15	300	8500	Não
113	Notebook Samsung Gamer	150	17500	Não
114	SSD 4TB	200	5750	Não
115	SSD 2TB	150	3750	Não
121	SSD 4TB	50	4150	Não
122	GAINWARD PHOENIX RTX3080ti	30	14150	Não
123	GAINWARD PHOENIX RTX3070	60	7399	Não
124	ECHO DOT ALEXA	1000	200	Não
125	Monitor Asus BK 35"	500	9990	Não



- **Funcionalidade READ:** Criação da página única de produtos:
 - Rota para acessar o método ***show()*** do controlador de ***Produtos***;
 - **Route::get('caminho/{parametro}', [ClasseController:class,'método']);**
 - **Método:** verbos http como ***get, post, put, patch, delete*** e ***option***;
 - **Caminho:** caminho da requisição (**URI**) para diferenciar rotas (***endpoint***);
 - **Parâmetro:** onde entrará a variável enviada na url, ex. ***ID, slug, etc...***
 - **Controlador:** nome da classe controladora seguido de ***::class***.
 - **Método:** nome do método a ser executado no controlador;

```
28 Route::get('/produto/{id}', [ProdutoController::class, 'show']);
```

CRUD com Laravel

Revisão da última aula – Funcionalidade READ do CRUD



- **Funcionalidade READ:** *Criação da página única de produtos:*
 - Para visitar a página única de um registro vamos acessá-la pelo seu **ID**
 - Portanto criaremos o método **show()** no controlador **ProdutoController**
 - O método recebe o **ID** como parametro e repassa ao método **find()** da classe de modelo que estende o **Eloquent**.

```
22     public function show($id)
23     {
24         return view('produto', [
25             'produto' => Produto::find($id)
26         ]);
27     }
```

CRUD com Laravel

Revisão da última aula – Funcionalidade READ do CRUD



- **Funcionalidade READ:** *Criação da página única de produtos:*
 - Criação do template que receberá e apresentará os dados do produto;
 - Exemplo de template de página única com blade: [Link](#).

```
resources > views > produto.blade.php > html > head
9  </head>
10 <body>
11     @if ($produto)
12         <h1>{{ $produto->nome }}</h1>
13         <p>{{ $produto->descricao }}</p>
14         <ul>
15             <li>Quantidade: {{ $produto->qtd_estoque }}</li>
16             <li>Preço: {{ $produto->preco }}</li>
17             <li>Importado: {{ $produto->importado ? 'Sim' : 'Não' }}</li>
18         </ul>
19     @else
20         <p>Produtos não encontrados! </p>
21     @endif
22     <a href="/produtos">#9664;Voltar</a>
23 </body>
24 </html>
```

CRUD com Laravel

Revisão da última aula – Funcionalidade READ do CRUD

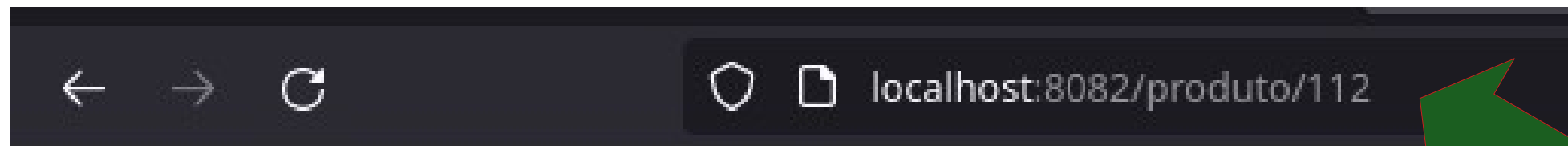


localhost:8082/produtos

Produtos



Id	Nome	qtd_estoque	preço
111	Samsung A5 - 2017	2	45
112	Notebook DELL Inspiron 15	300	85
113	Notebook Samsung Gamer	150	17
114	SSD 4TB	200	57
115	SSD 8TB	150	85



Notebook DELL Inspiron 15

I5 7600HQ 8GBMen GTX1030m SSD 1TB

- Quantidade: 300
- Preço: 8500
- Importado: Não

[voltar](#)

Deve voltar para a
lista de Produtos



- **Inserindo Dados no Banco: CREATE (CRUD) [Link](#) !**
- Usamos as classes da camada de Modelo.
- Métodos:
 - ***save()*** :
Salva os dados do objeto Model ***instanciado*** em memória no banco de dados.
Serão inseridos também os valores das colunas ***created_at*** e ***updated_at*** automaticamente.

Para usar o save o objeto modelo precisa estar instanciado com os valores de seus atributos correspondentes às colunas devidamente preenchidos.



- **Inserindo Dados no Banco: CREATE (CRUD)**
- Exemplo de uso do método **save()**:

```
28     public function store(Request $request)
29     {
30         $produto = new Produto;
31         $produto->nome = $request->nome;
32         $produto->descricao = $request->descricao;
33         $produto->qtd_estoque = $request->qtd_estoque;
34         $produto->preco = $request->preco;
35         $produto->importado = ($request->importado)?true:false;
36
37         if ($produto->save())
38         |     return redirect('/produtos');
39         else
40         |     dd("Erro ao inserir novo produto !");
41     }
```



- **Inserindo Dados no Banco: CREATE (CRUD)**
- Declarando a propriedade `fillable` na classe `model` poderemos usar os métodos ***fill()*** ou ***create()***:
 - ***fill()***: Preenche os atributos de um objeto já instanciado em memória diretamente, sem a necessidade de definir os atributos um-a-um. Recebe um ***array*** com os dados como parâmetro, mas **NÃO SALVA** no banco de dados;
 - ***create()***: **SALVA OS DADOS** no banco sem instanciar um objeto *model*. Também recebe um ***array*** como ***parâmetro***.
 - Exemplo da propriedade ***fillable*** na *model*, prevenção de ***mass assignment vulnerability***

```
8 class Produto extends Model
9 {
10     use HasFactory;
11     protected $fillable = ['nome', 'descricao', 'qtd_estoque', 'preco', 'importado'];
12 }
```



- **Inserindo Dados no Banco: CREATE (CRUD)**
- Exemplo usando *fill()* para preencher objeto a partir dos dados da requisição.

```
28 public function store(Request $request)
29 {
30     $produto = new Produto;
31     $array_input_request = $request->all();
32     $produto->fill($array_input_request);
33     $produto->importado = ($request->importado)?true:false;
34
35     if ($produto->save())
36     |     return redirect('/produtos');
37     else
38     |     dd("Erro ao inserir novo produto !");
39 }
```

Eloquent ORM - CREATE

Mapeamento Objeto-Relacional



- **Inserindo Dados no Banco: CREATE (CRUD)**
 - Simplificando tudo com o ***create()***.

```
28     public function store(Request $request)
29     {
30         $newProduto = $request->all();
31         $newProduto['importado'] = ($request->importado)?true:false;
32
33         if ([Produto::create($newProduto)])
34             return redirect('/produtos');
35         else
36             dd("Erro ao inserir novo produto !");
37     }
```

- **OBS:** Para usar ***fill()*** e ***create()*** é obrigatório declarar a propriedade ***\$fillable*** na Model!
- ***redirect()***: redirecionará para a página que lista todos os produtos!

Eloquent ORM - CREATE

Mapeamento Objeto-Relacional



- **Criando a Rota para a página de criação:**
 - Para utilizar o método **store** e armazenar um novo produto, primeiro vamos redirecionar o usuário para uma página com um formulário.
 - Arquivo **/routes/web.php**:

```
25 | #CRUD
26 | Route::get('/produto',[ProdutoController::class,'create']);
```

- No controlador **ProdutoController** criamos o método **create**, que basicamente irá redirecionar para um **template** com o formulário.

```
48 | public function create()
49 | {
50 |     return view('produto_create');
51 | }
```

- O arquivo **produto_create.blade.php** pode ser encontrado aqui: **Link !**



- **Formulário de inserção:**
 - O *Laravel* implementa segurança contra **Cross-Site-Request-Forgeries**.
 - Portanto no formulário é necessário usar um *token* gerado para cada sessão, que será validado pelo *Laravel* quando chegar a requisição (*Request*).
 - **Blade:** com o blade usamos facilmente a diretiva **@csrf**

```
11 <body>
12     <h1>Insert new Produto</h1>
13     <form action="/produto" method="POST">
14         @csrf
15         <table>
16             <tr>
17                 <td>Nome:</td>
18                 <td><input type="text" name="nome"/></td>
19             </tr>
20             <tr>
```


Eloquent ORM - CREATE

Mapeamento Objeto-Relacional



- **Formulário de inserção:**
- A diretiva no *blade* irá criar um campo do tipo *hidden* com o **token**:

```
<h1>Insert new Produto</h1>
▼ <form action="/produto" method="POST">
  <input type="hidden" name="_token"
    value="PQdYhWW7ahly5JPX1bMyydApv2GozNx0MnoazjSC">
  ► <table>⋮</table>
```

- Outra forma de gerar este token é criando diretamente o campo **input[hidden]** e acessando o token pela função **csrf_token()**

```
<h1>Insert new Produto</h1>
<form action="/produto" method="POST">
  {{-- @csrf --}}
  <input type="hidden" name="_token" value="{{csrf_token()}}"/>
  <table>
    <tr>
```



- **Criando a Rota para a requisição POST do Formulário:**
 - Agora criamos a rota para finalmente enviarmos uma requisição **POST** para o *Laravel* na URI **/produto**
 - Esta requisição será encaminhada para o método **store** da classe **ProdutoController**:

```
25  #CRUD
26  Route::get('/produto',[ProdutoController::class,'create']);
27  Route::post('/produto',[ProdutoController::class,'store']);
```

Route::post('caminho/da/rota',[ClassControladora, 'método'])

- **OBS:** O método invocado agora é **post()**, compatível com o método do formulário de criação de um novo produto!

Insert new Produto

Nome:

Descricao:

Quantidade em Estoque:

Preço:

Importado:

☐

Criar

[◀ Voltar](#)

Deve voltar para a lista de Produtos

Envia para a rota post("/produto")

Search HTML

```
<!DOCTYPE html>
<html lang="en">
  <head>...</head>
  <body data-new-gr-c-s-check-loaded="8.895.0" data-gr-ext-installed="">
    <h1>Insert new Produto</h1>
    <form action="/produto" method="POST">
      <input type="hidden" name="_token" value="Kp6AzQGePD9CPh37MUdBR3rKXoEy08Ejr27xrnEk">
    <table>
      <tbody>
        <tr>
          <td>Nome:</td>
          <td>
            <input type="text" name="nome">
          </td>
        </tr>
        <tr>...</tr>
        <tr>...</tr>
        <tr>...</tr>
        <tr>...</tr>
        <tr align="center">
          <td colspan="2">
            <input type="submit" value="Criar">
          </td>
        </tr>
        <tr align="center">
          <td colspan="2">
            <a href="/produtos" style="display: inline">◀ Voltar</a>
          </td>
        </tr>
      </tbody>
    </table>
  </body>
</html>
```

Eloquent ORM - UPDATE

Mapeamento Obejto-Relacional



- Como atualizar os dados com o *Eloquent*: UPDATE (CRUD) [Link](#)
- Podemos usar *find()* e *save()* para atulizar dados de um objeto no banco de dados. Após recuperar o objeto com o *find()*, alterar os valores de seus atributos enquanto instanciado e depois chamar o *save()*;

```
55 public function update(Request $request, $id)
56 {
57     $updatedProd = $request->all();
58     $produto = Produto::find($id);
59     $produto->nome = $updatedProd['nome'];
60     $produto->descricao = $updatedProd['descricao'];
61     $produto->qtd_estoque = $updatedProd['qtd_estoque'];
62     $produto->preco = $updatedProd['preco'];
63     $produto->importado = (isset($updatedProd['importado'])) ? true : false;
64     if (!$produto->save())
65     |     dd("Erro ao atualizar produto $id !");
66     return redirect('/produtos');
67 }
```

Eloquent ORM - UPDATE

Mapeamento Objeto-Relacional



- Como atualizar os dados com o *Eloquent*: UPDATE (CRUD) [Link](#)
- Utilizando diretamente o *update()*: Lembrando que para usá-lo precisamos declarar a propriedade protegida (*protected*) *\$fillable* no Model
- Neste exemplo se não fosse o campo booleano 'importado' o código ficaria ainda mais simples, apenas passando *\$request* → *all()* para o *update()*.

```
public function update(Request $request, $id)
{
    $updatedProduto = $request->all();
    $updatedProduto['importado'] = ($request->importado) ? true : false;
    if (!Produto::find($id)->update($updatedProduto))
    {
        dd("Erro ao atualizar produto $id !");
    }
    return redirect('/produtos');
}
```

/app/Http/Controllers/ProdutoController

Eloquent ORM - UPDATE

Mapeamento Objeto-Relacional



- **Criando página de atualização (edição): UPDATE (CRUD)**
- Vamos agora criar a rota para a página de atualização dos dados;
- A rota apontará para o método ***edit*** no controlador que redirecionará para a página com o formulário.

```
# - Update /routes/web.php
Route::get('/produto/{id}/edit',[ProdutoController::class,'edit'])->name('edit');
Route::post('/produto/{id}/update',[ProdutoController::class,'update'])->name('update');
```

- No exemplo já criamos as rotas para as páginas de edição (***GET***) e a rota do formulário ***POST***.
- Agora usamos o método ***name()*** para nomear as nossas rotas, assim poderemos referenciá-las pelo nome e não pelo ***URI***, quando necessário.
- **OBS:** É necessário chamar o ***find*** primeiro para localizar o registro!

Eloquent ORM - UPDATE

Mapeamento Objeto-Relacional



- Criando página de atualização (edição): UPDATE (CRUD)
 - Atualizamos a **view** do template *produtos.blade.php* para inserir os links.
 - Para gerar os links automaticamente usamos a função *route()* passando o nome que criamos para a *rota*:
 - Neste exemplo já estamos criando inclusive o link para a página que irá deletar o registro.

```
<td>
    <a href="{{route('delete',$produto->id)}}" title='Deletar'>#128465</a>
    <a href="{{route('edit',$produto->id)}}" title="Editar">✎</a>
</td>
```

/resources/views/produtos.blade.php

Eloquent ORM - UPDATE

Mapeamento Objeto-Relacional



- Redirecionando para a página de edição: UPDATE (CRUD)
 - A rota ***edit*** encaminha para o método ***edit()*** do controlador que redireciona para a página de edição com o formulário.
 - O template da página de edição pode ser encontrado aqui: ***Link***

```
63 public function edit($id)
64 {
65     $data = ['produto' => Produto::find($id)];
66     return view('produto_edit', $data);
67 }
```

/app/Http/Controllers/ProdutoController

- **OBS:** ***produto_edit*** é o nome do arquivo ***blade.php*** na página ***/resources/views/***.

Eloquent ORM - UPDATE

Mapeamento Objeto-Relacional



- **Formulário de edição: UPDATE (CRUD)**
 - A variável **\$data** com a chave **produto** enviada para a view será recuperada para preencher os campos do formulário:

```
<h1>Insert new Produto</h1>
<form action="{{route('update',$produto->id)}}" method="POST">
  @csrf
  <table>
    <tr>
      <td>Nome:</td>
      <td><input type="text" name="nome" value="{{ $produto->nome }}" /></td>
    </tr>
    <tr>
      <td>Descricao:</td>
      <td><textarea name="descricao" id="" cols="30" rows="10">{{ $produto->descricao }}</textarea>
    </tr>
    <tr>
      <td>Quantidade em Estoque:</td>
      <td><input type="text" name="qtd_estoque" value="{{ $produto->qtd_estoque }}" /></td>
    </tr>
  </table>
</form>
```

Enviar para o método **update()** de **ProdutoController**

/resources/views/produto_edit.blade.php

Eloquent ORM - UPDATE

Mapeamento Obejto-Relacional



localhost:8000/produto/111/edit

Insert new Produto

Nome: Samsung A5 - 2017

Descricao: Samsung A5 2017 2GB Exynos Octacore Dual Chip

Quantidade em Estoque: 10

Preço: 1600

Importado: ☐

Salvar Cancelar

Envia para a rota *update*

Deve voltar para a lista de Produtos? Não

Inspector Console Debugger Network Style Editor

Search HTML

```
<!DOCTYPE html>
<html lang="en">
  <head>...</head>
  <body data-new-gr-c-s-check-loaded="8.895.0" data-gr-ext-installed="">
    <h1>Insert new Produto</h1>
    <form action="http://localhost:8000/produto/111/update" method="POST">
      <input type="hidden" name="_token" value="Kp6AzQGePD9CPh37MUdBR3rKXoEy">
      <table>
        <tbody>
          <tr>
            <td>Nome:</td>
            <td>
              <input type="text" name="nome" value="Samsung A5 - 2017">
            </td>
          </tr>
          <tr>...</tr>
          <tr>...</tr>
          <tr>...</tr>
          <tr>...</tr>
          <tr align="center">...</tr>
        </tbody>
      </table>
```

Eloquent ORM - DELETE

Mapeamento Objeto-Relacional



- **Removendo um registro: DELETE (CRUD) Link !**
- Para remover registros do banco de dados usamos os métodos:
- **delete()**: exclui do banco o registro referente ao objeto instanciado na memória, portanto deverá ser usando com o **find()** para primeiro encontrar o objeto pelo seu **ID** e depois removê-lo.

```
51     public function delete($id)
52     {
53         if([Produto::find($id)->delete()])
54             return redirect('/produtos');
55         else dd($id);
56     }
```

/app/Http/Controllers/ProdutoController

Eloquent ORM - DELETE

Mapeamento Objeto-Relacional



- **Removendo um registro: DELETE (CRUD)**
- Para usar o ***delete()*** criamos a seguinte rota:

```
Route::get('/produto/{id}/delete',[ProdutoController::class,'delete'])->name('delete');  
/routes/web.php
```

- Na view basta ter o link gerado pela função ***route()***:

```
<td> /resources/view/produtos.blade.php  
<a href="{{route('delete',$produto->id)}}" title='Deletar'>#128465</a>
```

- **OBS:** Neste caso ao clicar no link o item será diretamente removido, pois será chamado o método ***delete()*** do slide anterior, sem nenhuma confirmação por parte do usuário e por uma requisição do tipo ***get***.



- Removendo um registro: DELETE (CRUD)

ENVIA PAR A ROTA DELETE:
O QUE EXECUTARÁ O MÉTODO DELETE
EXCLUINDO IMEDIATAMENTE O REGISTRO

Produtos

[⊕ Novo](#)

Id	Nome	qtd_estoque	preco	Importado	Ações
111	Samsumg A5 - 2017	10	1600	Sim	 
112	Notebook DELL Inspiron 15	300	8500	Não	 
113	Notebook Samsumg Gamer	150	15500	Sim	 
114	SSD 4TB	200	5750	Não	
115	SSD 2TB	150	4750	Não	

Eloquent ORM - DELETE

Mapeamento Objeto-Relacional



- **Removendo um registro: DELETE (CRUD)**
 - **destroy()**: exclui do banco o registro referente à chave primária **{ID}** repassado como parâmetro, não é necessário usar o método **find()**.

```
70     public function remove(Request $request, $id)
71     {
72         if($request->confirmar=="Deletar")
73             if(!Produto::destroy($id))
74                 dd("Erro ao deletar o produto $id !");
75         return redirect('/produtos');
76     }
```

/app/Http/Controllers/ProdutoController

- **OBS:** Diferente do primeiro exemplo do delete, este usa uma **Request** e verifica se existe a variável **confirmar** com o valor “**Deletar**”, antes de proceder com o processamento da remoção pelo método **destroy()**.



- **Removendo um registro: DELETE (CRUD)**
 - **CONFIRMAÇÃO:** Para se ter uma página de confirmação da remoção antes de realmente remover o registro, vamos criar duas rotas, uma para apresentar detalhes do registro e o formulário de confirmação da remoção e a outra rota para a requisição do tipo **POST** para o método que irá realmente processar a remoção no banco de dados:

```
36 # - Delete /routes/web.php
37 Route::get('/produto/{id}/delete',[ProdutoController::class,'delete'])->name('delete');
38 Route::post('/produto/{id}/delete',[ProdutoController::class,'remove'])->name('remove');
```

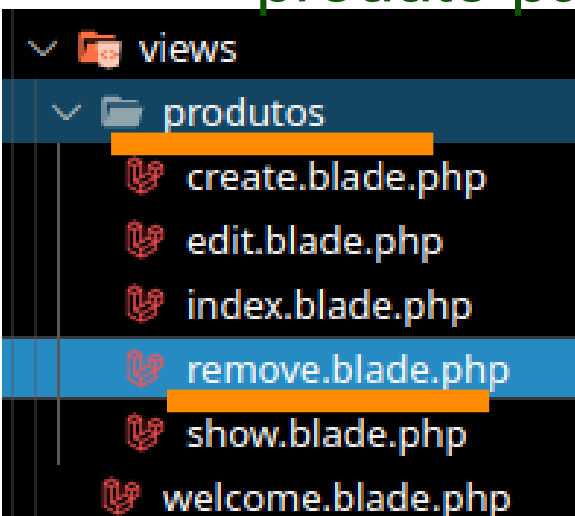
- **GET:** Página de confirmação da remoção, template aqui: **Link !**
- **POST:** Rota para o método **remove** do controlador, slide anterior!

Eloquent ORM - DELETE

Mapeamento Objeto-Relacional



- **Removendo um registro: DELETE (CRUD)**
 - Agora o método ***delete*** irá apenas redirecionar para a página única com os dados do produto e o formulário de confirmação da remoção.
 - **produto.remove**: é o nome do template da página única. O ponto (.) determina um diretório, ou seja, o template está em produtos/remove.blade.php
 - **'produto'**: é a chave do array associativo que transportará os dados do produto para a **view**.



```
public function delete($id)
{
    return view('produtos.remove', [
        'produto' => Produto::find($id)
    ]);
}
```



- Removendo um registro: DELETE (CRUD)

ENVIA PAR A ROTA DELETE:
PÁGINA DE CONFIRMAÇÃO

Produtos

[⊕ Novo](#)

Id	Nome	qtd_estoque	preco	Importado	Ações
111	Samsung A5 - 2017	10	1600	Sim	 
112	Notebook DELL Inspiron 15	300	8500	Não	 
113	Notebook Samsung Gamer	150	15500	Sim	 
114	SSD 4TB	200	5750	Não	 
115	SSD 2TB	150	4750	Não	 



- Removendo um registro: DELETE (CRUD)

```
<form action="{route('remove',$produto->id)}" method="post">  
  @csrf  
  /resources/view/produtos_remove.blade.php
```

Samsung A5 - 2017

Samsung A5 2017 2GB Exynos Octacore Dual Chip

- Quantidade: 10
- Preço: 1600
- Importado: Sim

Confirmar a exclusão deste item?

Deletar

Cancelar

```
if($request->confirmar=="Deletar")  
    if(!Produto::destroy($id))  
        dd("Erro ao deletar o produto $id !");  
    return redirect('/produtos');
```

REDIRECIONA PARA A ROTA
"PRODUTOS"

ENVIA PARA A ROTA REMOVE:
EXECUTA O MÉTODO DO
CONTROLADOR



1) Criar CRUD para três tabelas do banco de dados do projeto/TCC:

- Utilize os exemplos vistos nesta aula e implemente o sistema de CRUD seguindo as especificações de uso do *Laravel*.
- Adicione na página única de cada registro o link para a edição e remoção daquele registro.

Entregue o projeto em uma pasta compactada no **moodle** com o padrão:

- ***nome_do_aluno_cstsi_daoo.zip***

Ou se preferir pode enviar um arquivo com o *link* do repositório remoto (github, gitlab, bitbucket etc...) para clone.

OBS: Salve na pasta ***database/dump*** um dump do seu banco de dados com os dados, não apenas o sql de criação.